

WAPH - Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Team Project

Project Topic/Title

Team Members

1. Omkar Sontake, sontakor@mail.uc.edu
2. Akshat Chaturvedi, chaturat@mail.uc.edu

Project Management Information

- Source code repository (private access): GitHub Repository
- Project homepage (public): Project Homepage
- Demo video link : <https://youtu.be/6aQlykx2RT0>

Revision History

Date	Version	Description
02/07/YYYY	0.0	Init draft
02/12/2024	0.1	Sprint 1.0
02/21/2024	0.2	Sprint 2.0
02/31/2024	0.3	Sprint 3.0

Overview

We started doing the project by first creating an SSL key and setting up the HTTPS and team local domain name. In addition to that, we also updated the index.html file for the basic personal portfolio which includes our website links and our introduction. In Sprint 1, we focused on database design and implementation such as user registration and basic login features. We also added a functionality for logged in users to browse their posts which are available on database, and change their username and password. We created database management schemas in which we had selected database management systems and set up tables to store users details, posts and comments. We ensured that the credentials were securely stored and there was appropriate server-side validation for user registration and login. Furthermore, we added a functionality so that logged in users can view posts and change their profiles. We held regular team

meetings every week to discuss our progress as well as the things which need to be improvised further.

System Analysis

Problem Definition

We created a web application by adding the edit,delete and comment functions on posts with user management. For sprint 1, we simply created a design and implemented it into the database. In addition to this, we added functionalities for user registration,login,change password,edit profile, so that users can view their posts from the database. In Sprint 2, we enhanced this functionality by which logged in users can add additional posts,comment on the posts and edit their own posts. We also added access controls so that unauthorized modifications can be prevented. For Sprint 3, we added a superuser functionality in which we added a feature to allow superusers to see all the users and disable the accounts which they want to.

System Modeling

The main goal in developing the project's system model is to develop all the functions which are required for the applications. In order to achieve this, first we have to design the database structure which includes tables for users,superusers,comments and posts. All these functions in the database structure are connected to each other. The thing which we need to keep in mind is to choose the right structure for the database. We have to maintain the data integrity and scalability as well. Furthermore, we implemented server-side validation and secured password hashing for user registration and login features to ensure safe authentication. In addition to this, the users are given the ability to give a post,delete post,edit profile and change password while they are logged in.

Security Implementation

In the last sprint, we have did a security implementation. In this implementation, we protected the user data and the system integrity. In order to protect the password which are stored in the database, we have to implement secure user authentication methods like hashing. Furthermore, we used server-side validation to ensure the user input is safe and clear from the vulnerabilities and attacks such as SQL injection and cross-site scripting attacks (XSS). Only the authorized users can change their profile details and gain access to the secured part of the system.

High-level Requirements

User Registration and Authentication:

- Users should be able to register for an account.
- Users should be able to log in and log out securely.
- Users should be able to change their password.

User Profile Management:

- Users should be able to edit their profile information, including name, additional email, and phone number.

Content Management:

- We want users to be able to view posts from the database.
- Additionally the logged-in users should be able to add new posts, comment on any post, edit their post and delete their post.

Access Control:

- Users won't be able to edit or delete posts made by other users.
- A superuser should be able to disable or enable user accounts.

System Design

We used an approach that aligns with the design structure. We established some foundational components in sprint 1 which includes a secure database schema to handle user data, posts, registration and user authentication. We made sure that users can securely log in and log out and also manage their passwords.

In sprint 2, we enhanced user interaction with the system by which the user can create, view and interact in the content. We have created additional functionalities for additional posts, commenting on the posts and edit or delete them. We ensured security so that the user can only modify their own content.

In Sprint 3, we created a superuser to enable and disable the user accounts, ensuring better management of the system.

Use Case Realization

Our goal for the features of this application are to finish the following tasks - Registering as a new user, logging in, altering their profile, changing their password, and viewing their post activity. We generated two PHP files for each task. A front-end task and a back-end application task are separated.

For a uniform UI user experience, we used the CSS styles in all our php files.

Database Design and Implementation: It is used for the requirements of the application and for designing a database structure that can accommodate the required data into it. We used MySQL we made the tables for users, posts and the other required fields ensuring that they are correctly connected to one another and are given a proper.

User Registration and Login: We have developed and implemented the functionalities for user registration and user login. We created forms where users can sign up and login. The sign up, the user can enter their information such as email, password, etc. We used server-side validation to ensure the data's security and integrity. To enable the user authentication, we incorporated the session management and login functions.

Once the user has signed in, they will have the following options -

Change Password: We created a form where the user can change their old password to a new one. To ensure the safety and privacy of the user, we implemented server-side input validation.

Edit Profile: We created a page where user who is logged in can edit their profile details such as email address, phone number and all the other relevant information which they want to change.

Retrieve posts from database: The user can fetch and extract information about the post from the database. We provided users with postings that are easy for them to navigate. The user can store as much posts as they want to store.

Database

Sprint 1: We established the database infrastructure by creating a user `waph_team13` and a database named `waph_13`. Within this database, we created several tables: `users`, `chat_messages`, `comments`, `posts`, `profiles`, `superusers`, and `users`. These tables were designed to handle the core functionalities of user management, content storage, and interaction.

source code: `database-data.sql`

```
CREATE TABLE users(  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50),  
    password VARCHAR(255) NOT NULL,  
    is_disabled BOOLEAN DEFAULT FALSE  
);  
CREATE TABLE profiles(  
    user_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    additional_email VARCHAR(255),  
    email VARCHAR(255) UNIQUE NOT NULL,
```

```

        phone VARCHAR(20),
        FOREIGN KEY (user_id) REFERENCES users(user_id)
    );

CREATE TABLE posts(
    post_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE comments(
    comment_id INT PRIMARY KEY AUTO_INCREMENT,
    post_id INT,
    user_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES posts(post_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE superusers(
    superuser_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);

CREATE TABLE chat_messages(
    message_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_id INT,
    receiver_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sender_id) REFERENCES users(user_id),
    FOREIGN KEY (receiver_id) REFERENCES users(user_id)
);

INSERT INTO superusers (username,password) VALUES ('admin',md5('admin'));
INSERT INTO users (username,password) VALUES ('admin',md5('admin'));

GRANT ALL ON users TO 'waph_team13'@'localhost';
GRANT ALL ON profiles TO 'waph_team13'@'localhost';
GRANT ALL ON posts TO 'waph_team13'@'localhost';
GRANT ALL ON comments TO 'waph_team13'@'localhost';
GRANT ALL ON superusers TO 'waph_team13'@'localhost';
GRANT ALL ON chat_messages TO 'waph_team13'@'localhost';

```

Sprint 2: In this sprint, we used the ‘posts’ table to manage the content which the user has created. The table was structured to store the user who had made the post, created it’s content and also record the time at which the post was created by the user. It enables effective tracking and display of posts within the application.

Sprint 3: We used the `superusers` and `users` tables to implement an access control mechanism for the superuser functionality. This ensured that only verified superusers could access the superuser page, the non-superusers are redirected to the login page. Successful login was only permitted if the user was identified as a superuser, adding a crucial layer of administrative control to the system.

User Interface

Sprint 1: We utilized HTML and CSS to build a simple user interface. This UI interface was designed to be accessible, ensuring that users could easily navigate and interact with the application.

Sprint 2: Continuing from Sprint 1, we used HTML and CSS to develop the add new post page. This page was designed to maintain the simplicity and accessibility established earlier, allowing users to easily add new content.

Throughout the application, we have used HTML, JavaScript, and CSS to style the application. We carefully arranged elements to ensure that the website was responsive and easy to navigate, providing a seamless user experience.

Development Approach

Our team adopted a structured development approach, utilizing PHP, HTML, CSS, and SQL across all sprints. We placed a strong emphasis on security and best practices, including the exclusive use of prepared statements to prevent SQL injection, enforcing HTTPS, and hashing passwords before storing them in the database.

Sprint 1: During this phase, we focused on setting up the important functionalities, such as user registration and authentication. We used PHP, HTML, and CSS to build the user interface, and SQL for database interactions. We faced difficulties with database configuration, so all the work was done on Omkar’s laptop. This required us to meet in person to individually complete our tasks on the same computer. After building each functionality, we thoroughly tested the web pages to ensure that everything is working as expected.

Sprint 2: In this sprint, we addressed the issues which we encountered in Sprint 1 and successfully completed the tasks on time. We have used HTML and CSS for front-end development, particularly for the add new post forms. On the backend, we utilized SQL and PHP to store and retrieve data. By efficiently sending data to the posts table, we enabled the application to display posts

made by users. Immediate testing of our solutions allowed us to quickly identify and resolve any issues, ensuring the application functioned correctly.

Sprint 3: We further refined our development process by maintaining the use of PHP, HTML, CSS, and SQL. This sprint focused on enhancing administrative capabilities by implementing superuser functionalities. We ensured that only verified superusers could access certain pages, redirecting non-superusers to the regular user login page. Throughout this sprint, we continued our practice of immediate and thorough testing to ensure all new functionalities were robust and secure.

Overall, our development approach emphasized collaboration, security, and continuous testing, allowing us to build a reliable and user-friendly application.

Security Analysis

In our project, we applied several security programming principles to ensure the protection and integrity of the application and user data.

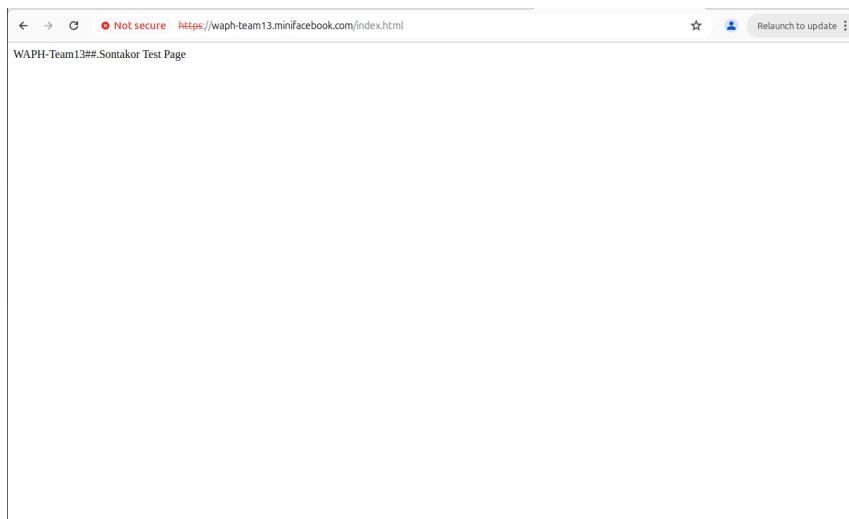
- **Security Programming Principles:** We used the prepared statements in SQL queries to prevent SQL injection attacks. We used hashing for storing the passwords for the security. We used HTTPS across the application to protect the data which is transmitted between the client and server. We also implemented input validation to prevent Cross-Site Scripting (XSS) attacks.
- **Database Security Principles:** Our database design included access controls and prepared statements to defend it against SQL injection. We made sure that the sensitive data, such as passwords, was hashed. For more security, we used regular database checkups and encryption.
- **Defense Mechanism:** We implemented thorough error handling and input validation to defend it. We used the validation checks and catch blocks to handle the unexpected inputs and system errors maintaining the stability and security of the system.
- **Defense Against Known Attacks:** We defended against common attacks as follows:
 - **XSS:** Input fields were sanitized and validated to prevent malicious scripts from being executed in the browser.
 - **SQL Injection:** Prepared statements and parameterized queries were used to prevent unauthorized access to the database.
 - **CSRF:** We implemented anti-CSRF tokens to prevent cross-site request forgery attacks, ensuring that requests originate from authenticated users.
 - **Session Hijacking:** We used secure session management practices, including setting secure and HTTP-only flags on cookies to protect

session data.

- **Role Separation:** We separated the roles of superusers and regular users by giving regular users different permissions to superusers. Superusers have more privileges and access to administrative functions such as enabling and disabling accounts, while regular users have access to their own content and profile management.
-

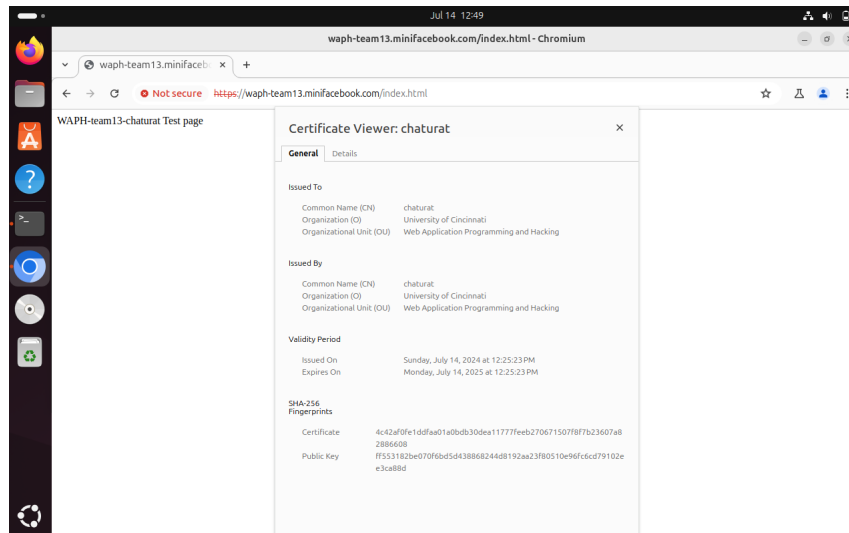
Demo (screenshots

Sprint0



Sprint 0: Screenshot 1





Sprint 0: HTTPS Deployment

Frontend We used HTML, CSS and Javascript functionalities in the PHP files for the frontend user interface. HTML was used to structure the user interface and define elements like buttons, forms and navigation menus. We used CSS to give styles to the page giving it an attractive appearance and layout so that it can be used across various screens and devices. We used Javascript for enhancing the user interaction by enabling features like interactive menus, form validation and updates without refreshing the page. We used HTML and Javascript to implement client-side validations and improved usability by providing feedback on the errors, thus enhancing the user experience.

Sprint 1

```
mysql> describe users;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| user_id    | int       | NO   | PRI | NULL    | auto_increment |
| username   | varchar(50) | NO   |     | NULL    |                |
| password   | varchar(255) | NO   |     | NULL    |                |
```

Sprint 1: Users Table

```
mysql> describe posts
-> ;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| post_id | int | NO | PRI | NULL | auto_increment |
| user_id | int | YES | MUL | NULL | |
| content | text | YES | | NULL | |
| timestamp | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Sprint 1: Posts Table

WAPH-Login page x Sign Up

Jul 21 13:05

https://waph-team13.minifacebook.com/form2.php

Please Login

Team 13

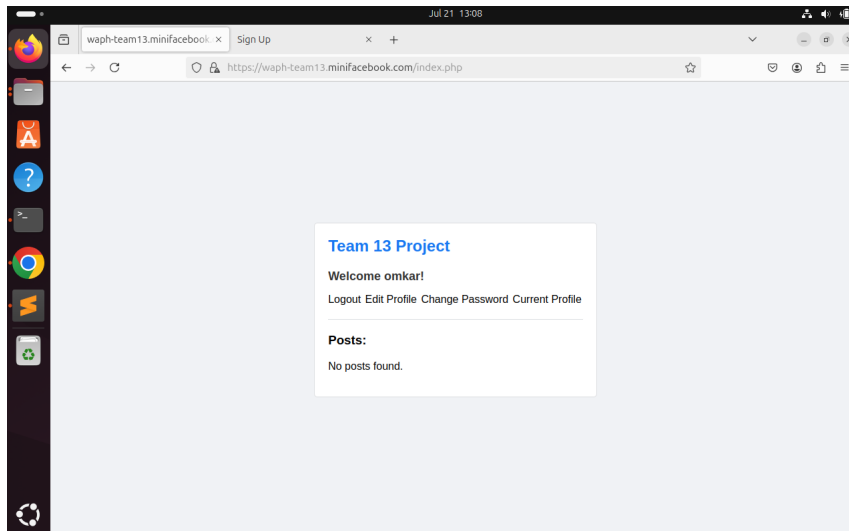
Username: omkal

Password:

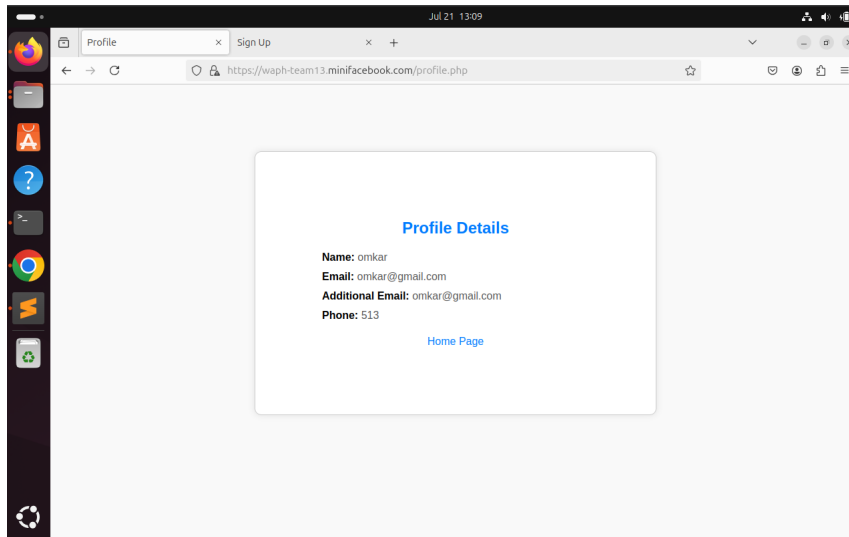
Log In

Sign Up

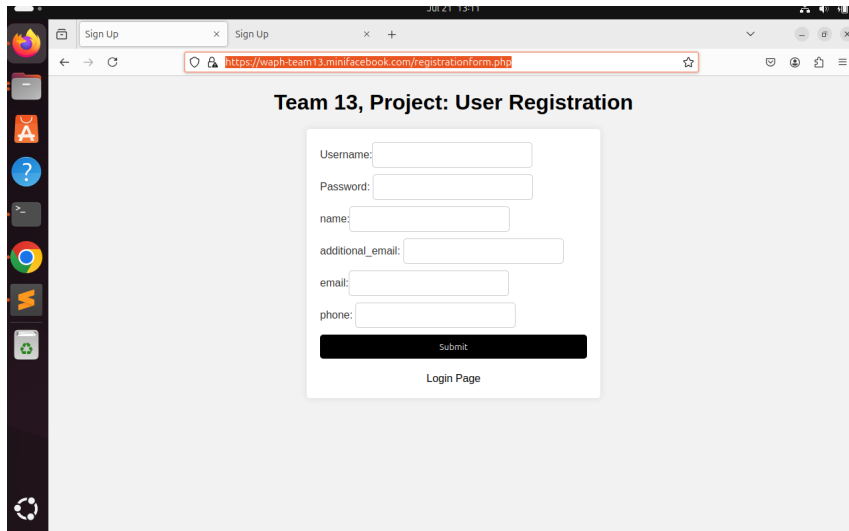
Sprint 1: Login page



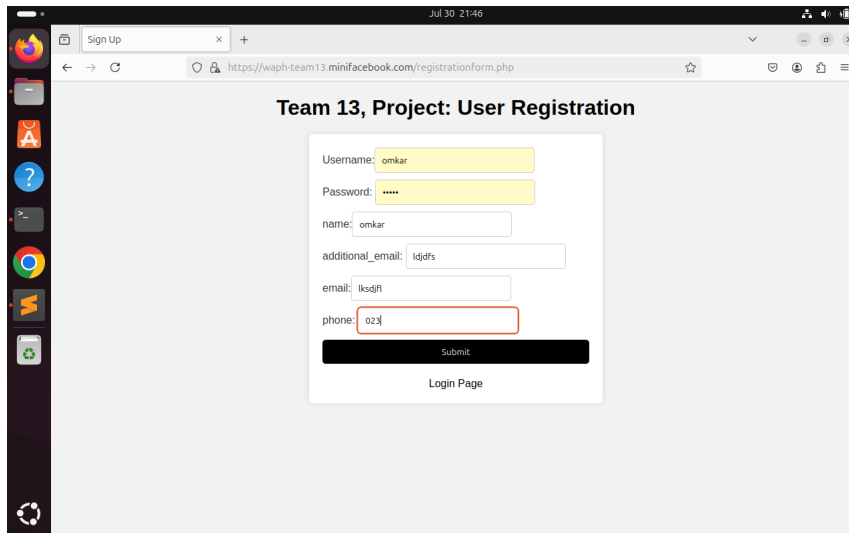
• Sprint 1: Home Page(After logging in)



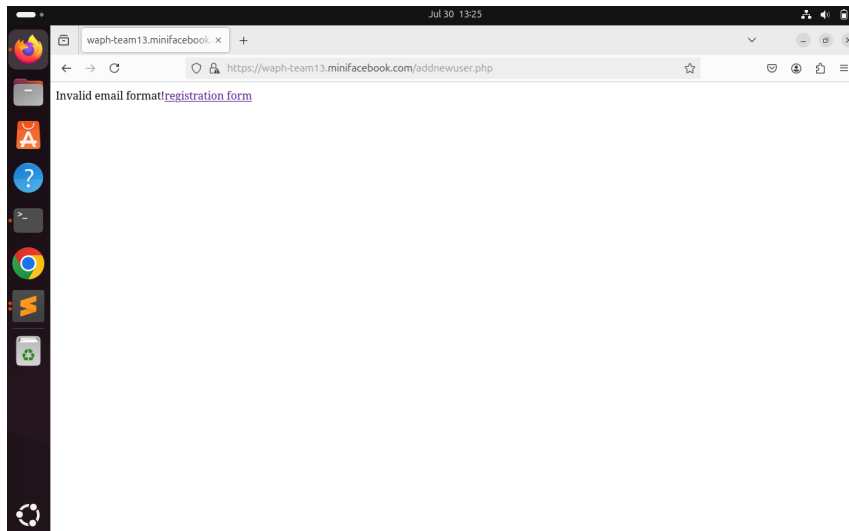
• Sprint 1: Profile Details Page



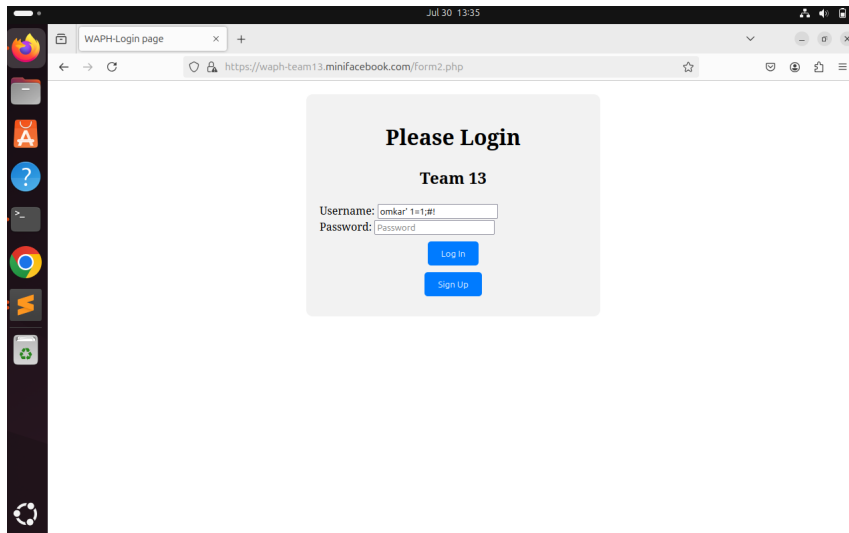
Sprint 1: Register User Page



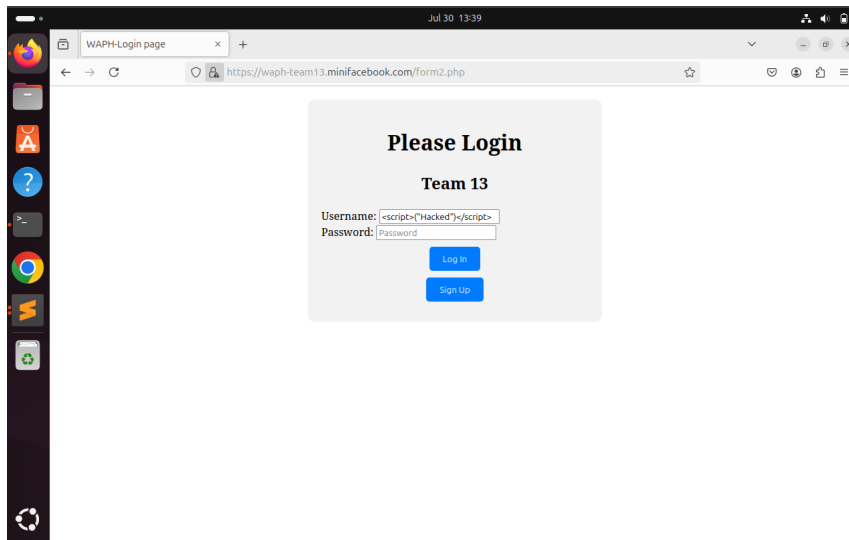
Sprint 1: User validation



• Sprint 1: Errothrown After Invalid email format

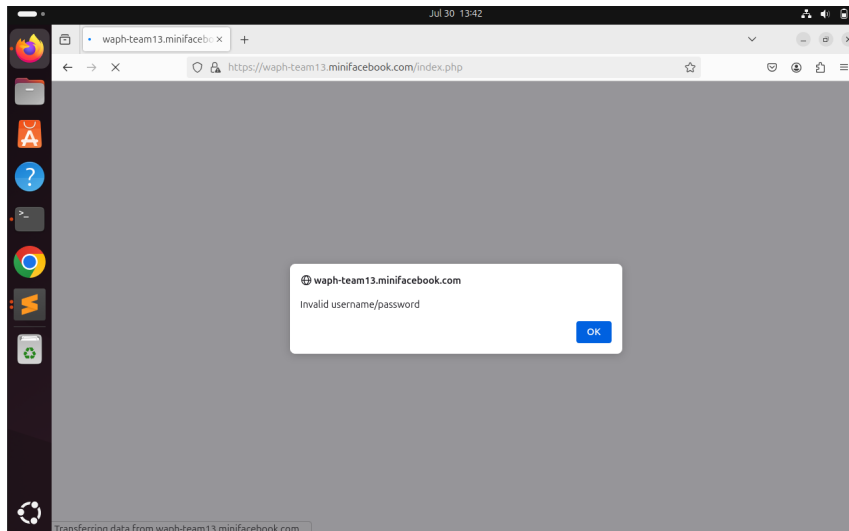


• Sprint 1: User validation

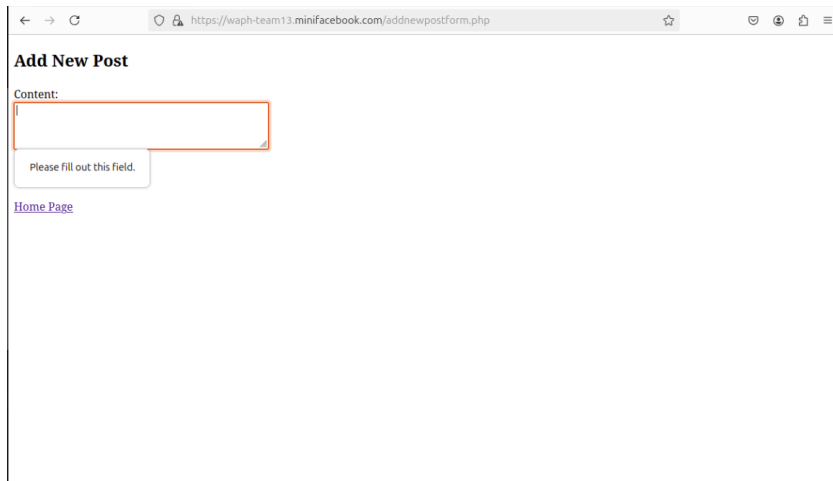


Sprint 1: Another version of XSS attack

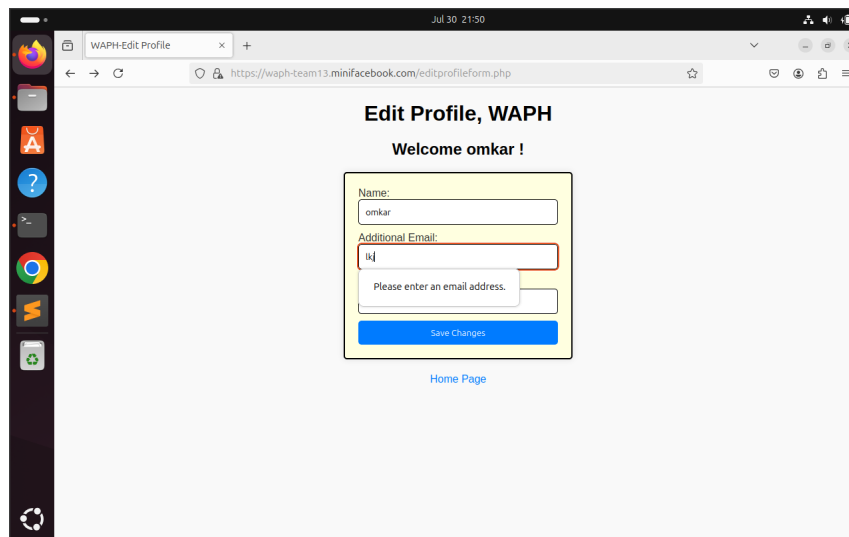
We created a secure login and register user page by establishing a secure database to store user details, including encrypted passwords and usernames. For the user sign-up process, we developed a registration form where users can enter their username, password, and any additional required data. The information which the user enters is stored in the database, ensuring that passwords are hashed before they are stored in the database. We also designed a login form for the user so that they can enter their credentials and if they match with the database, the user can successfully login. To maintain user state across application pages, we implemented session management by storing user information, such as user ID and username, in session variables. We used the Access control methods to restrict access to certain sites or features based on user login status. The unauthorized users are redirected to the login page. Furthermore, we provided a logout option for users to end their session securely and they can only revisit their page after logging in again. Lastly, I implemented error handling protocols to display appropriate error messages for failed login attempts such as invalid email format, incorrect information and other errors. This approach ensures the security and proper management of user information while providing an interactive user experience.



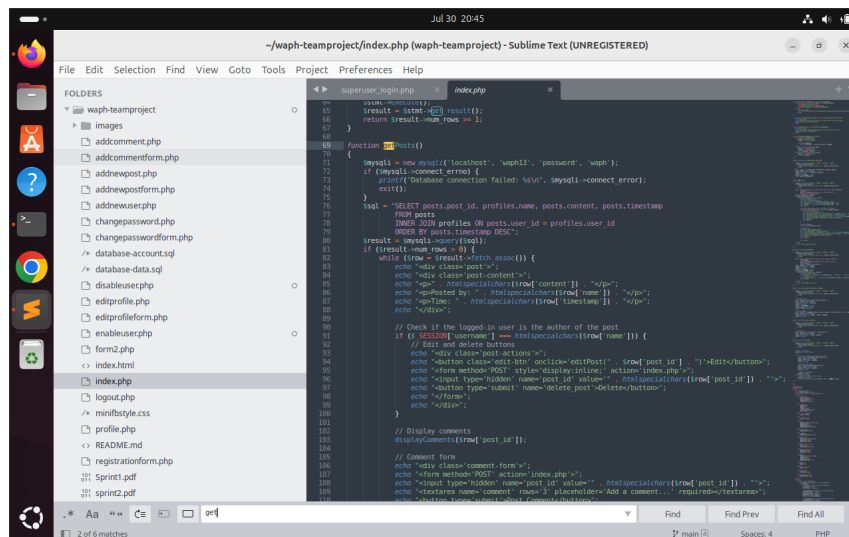
• **Sprint 1: Error thrown from attacks above.**



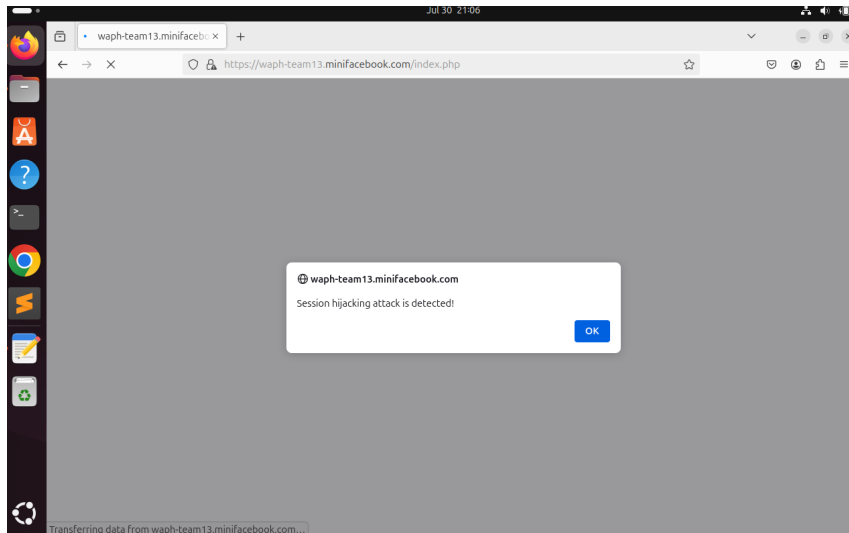
• **Sprint 2: Posts page filter**



Sprint 2: Input sanitization

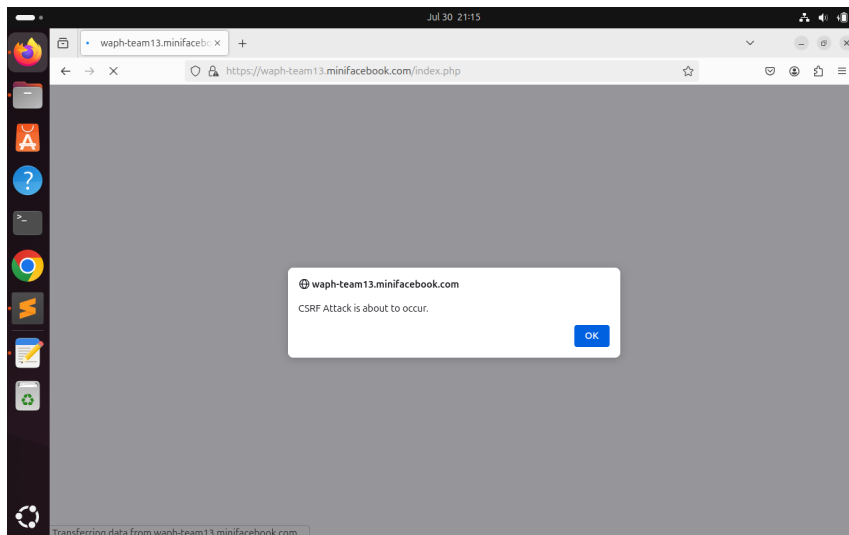


Sprint 2: Code for security



• Sprint 2: session hijacking

- Lab 1 Sprint 2: Code to prevent Session hijacking



• Sprint 2: CSRF attack

Attacks For preventing the session hijacking, we created a unique SSL certificate with our required information in it so that the website can be operated on HTTPS, and also enhancing its security. The `session_destroy()` function prevents the transfer of the cookie across different browsers. To protect the user credentials and session identifiers from unauthorized cookies, we used the effective session management. We used measures such as validating sessions post, generated session IDs and securing client side server communication with HTTPS. To

achieve more security, we used secure session cookies with HTTP and secure attributes and stored all the relevant data on the server.

We used the `/var/log/apache2/access.log` command to ensure that the cookies are stored in a correct way.

I implemented Cross-Site Request Forgery (CSRF) so that it can be unique for each user session. We added these tokens to forms and verified them upon submission for ensuring the validity of the request. With these tokens, the application blocks all the CSRF attacks and prevents any other user from copying the token. More implementations such as 'SameSite' cookie attribute further prevents the application from the CSRF attacks.

We used the 'SameSite' attribute to limit the cookies to work in the website. These cookies reduce the risk of stealing of the cookies from one browser to another and generating any unrelated requests.

```

Database changed
mysql> select * from waph;
ERROR 1146 (42502): Table 'waph.waph' doesn't exist
mysql> select * from users;
+-----+-----+-----+-----+
| user_id | username | password | is_disabled |
+-----+-----+-----+-----+
| 1 | admin | 68115705108ddd6f7ba9a458bb175363 | 0 |
| 2 | onkar | 68115705108ddd6f7ba9a458bb175363 | 0 |
| 3 | akshat | 91d3585c4023ba351e6aa8d94289a3f0 | 0 |
| 4 | onkartest | 68115705108ddd6f7ba9a458bb175363 | 0 |
| 5 | test | 098f6bcd4621d373cade4e832627b4f6 | 0 |
| 6 | video | 421b47ffd946ca883b65cd668c6b17e6 | 0 |
| 7 | user1 | $2y$10$CSX0nHkqNZR102evseQ9K0v6Z0quN23AKhN/8rLqv0z.L550WaoDC | 0 |
| 8 | user2 | 7e58d63b69197ceb55a1c487989a3720 | 0 |
| 9 | user3 | 92877af70a45f86a2ed7fe81e1236b78 | 0 |
| 10 | Akshat6115 | e19d5cd5af8378da05f63f891c7467af | 0 |
| 11 | user4 | 3f02ebe3d7929b091e3d8ccfde2f3bc6 | 0 |
+-----+-----+-----+-----+
11 rows in set (0.00 sec)

mysql> select * from superusers'
';
'; ^C
mysql> select * from superuserss;
ERROR 1146 (42502): Table 'waph.superuserss' doesn't exist
mysql> select * from superusers;
+-----+-----+-----+
| superuser_id | username | password |
+-----+-----+-----+
| 1 | admin | 5f4dcc3b5aa765d61d8327deb882cf99 |
| 2 | superuser | 0baae2f0ae20150db78f58cdac442a9 |

```

Sprint 2: Databases password hashing

Database I developed a passkey and built the database structure such that it could be accessible with it. The database is set up to securely store user information, reducing the possibility of data breaches or unwanted access. Making tables, fields, and connection is a necessary step in accurately specifying the data model. Secure procedures are followed while interacting with the database; for example, prepared statements and parameterized queries are used to prevent SQL injection attacks. Passwords and other sensitive data are also hashed before being entered into the database.

```
$mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
```

Team 13, Project: User Registration

Username: test

Password:

name: test

additional_email: test@gmail.com

email: test@gmail.com

phone: 513

Submit

[Login Page](#)

Sprint 1:Demo: user registering

Profile Details

Name: test

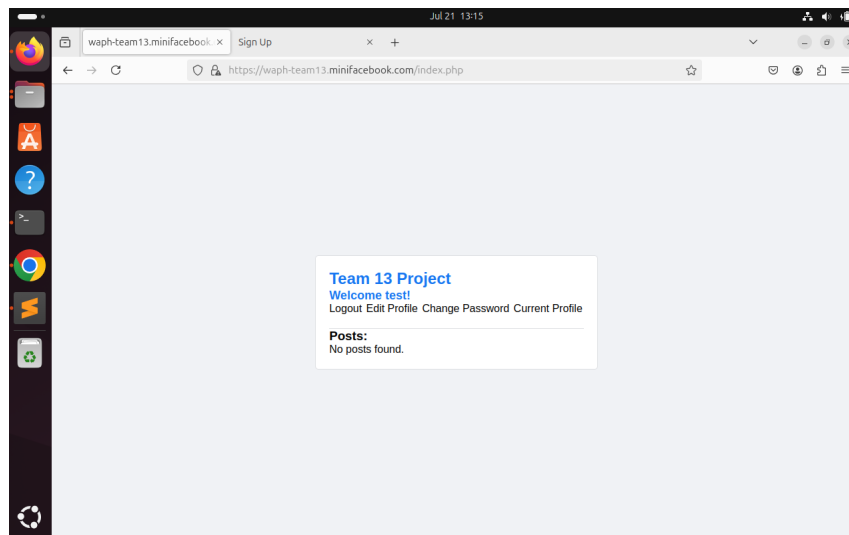
Email: test@gmail.com

Additional Email: test@gmail.com

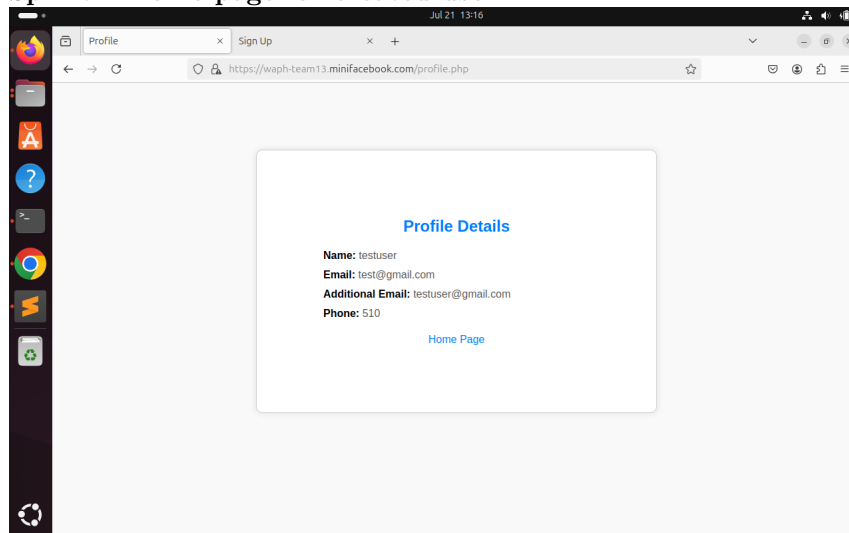
Phone: 513

[Home Page](#)

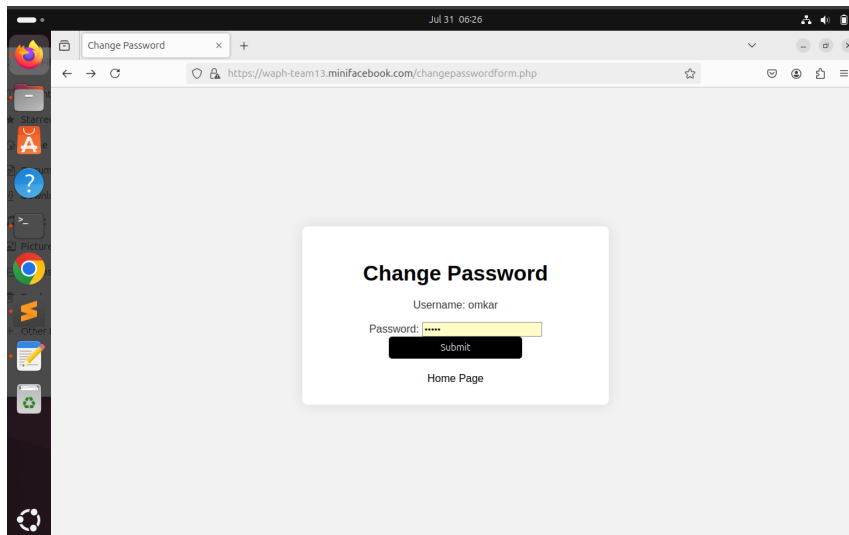
Sprint 1:Demo: User details displayed in profile page



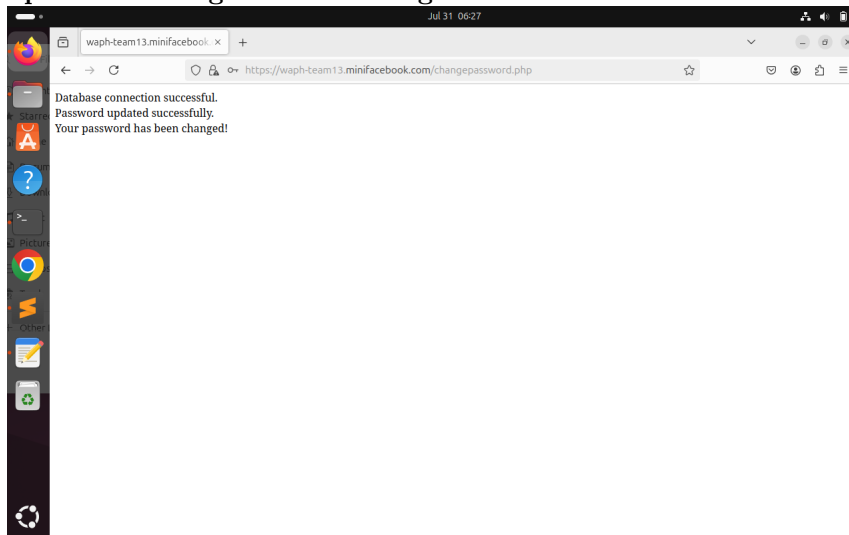
• Sprint 1: Home page for created user



• Sprint 1: Users details for change.(To show edit profile and change password functionality)



Sprint 2: Change Password Page



Sprint 2: CHange Password Success

Sprint 2



← → ↻ https://waph-team13.minifacebook.com/addnewpostform.php ☆ 🔒 📄

Add New Post

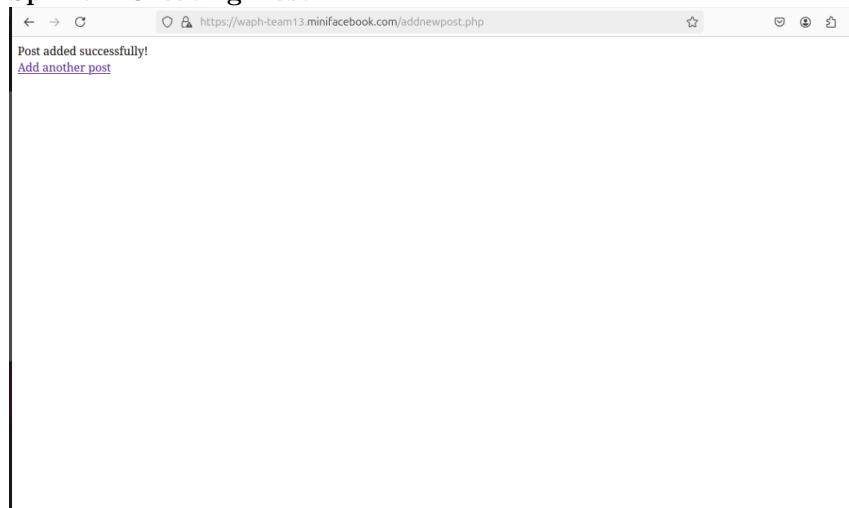
Content:

This is the first post on the application

Submit

[Home Page](#)

- ### Sprint 2: Creating Post

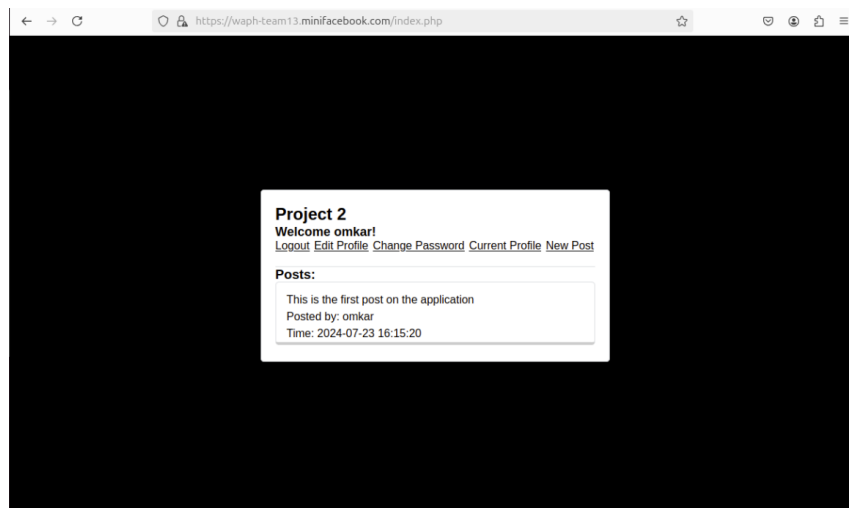


← → ↻ https://waph-team13.minifacebook.com/addnewpost.php ☆ 🔒 📄

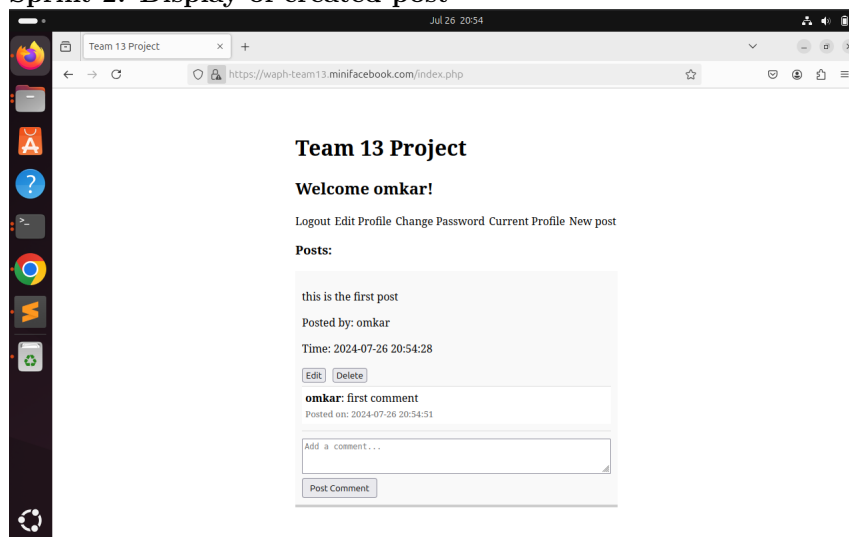
Post added successfully!

[Add another post](#)

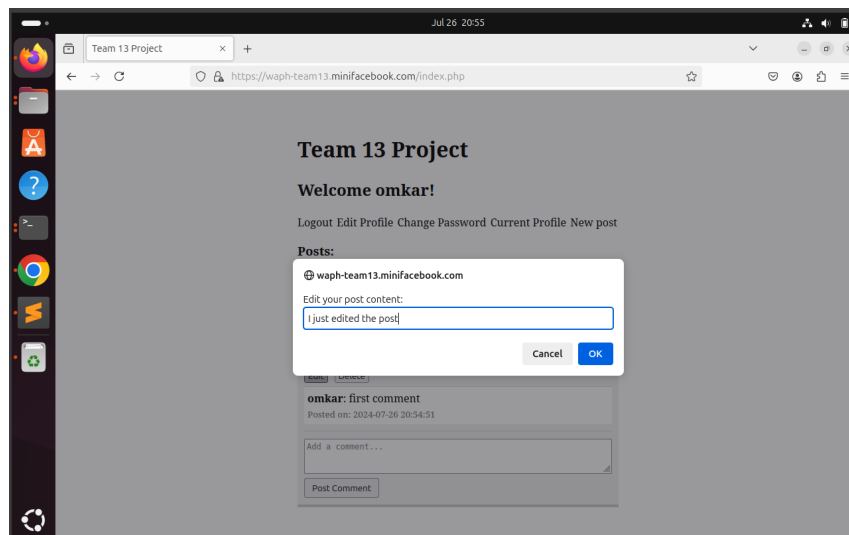
- ### Sprint 2: Post creation success



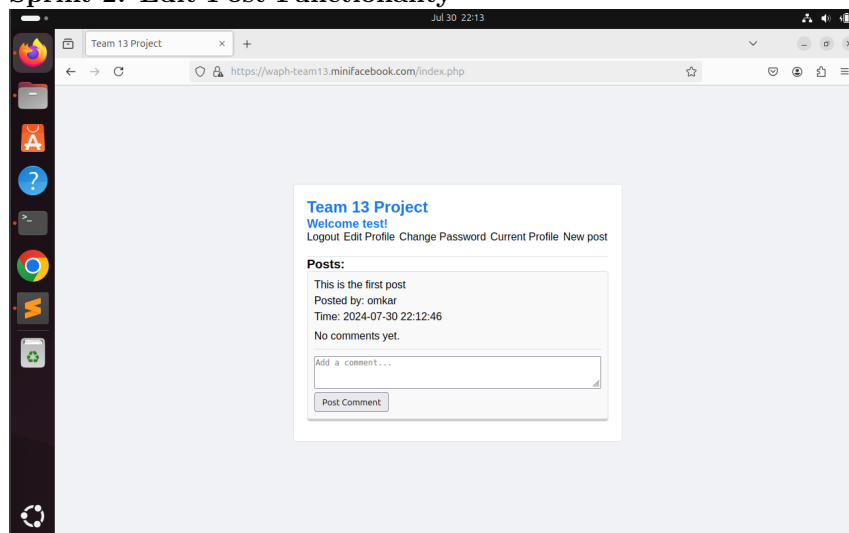
Sprint 2: Display of created post



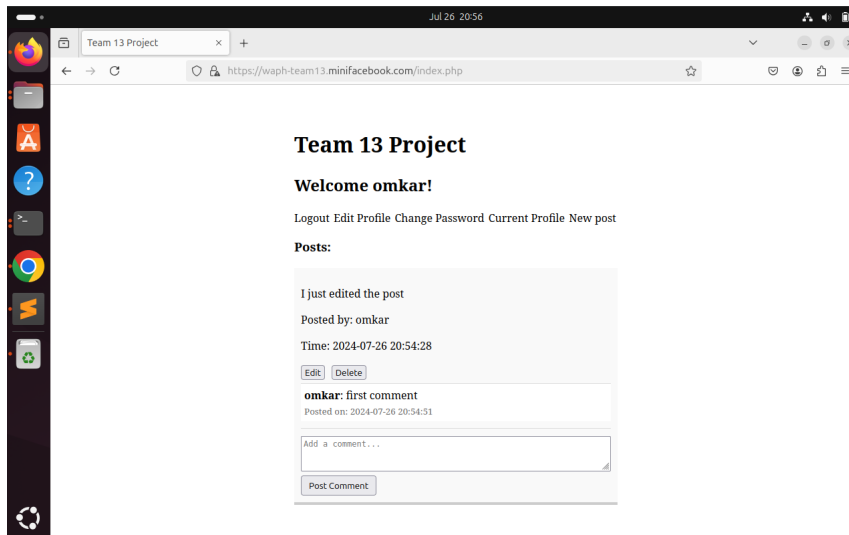
Sprint 2: Add and delete functionality addition



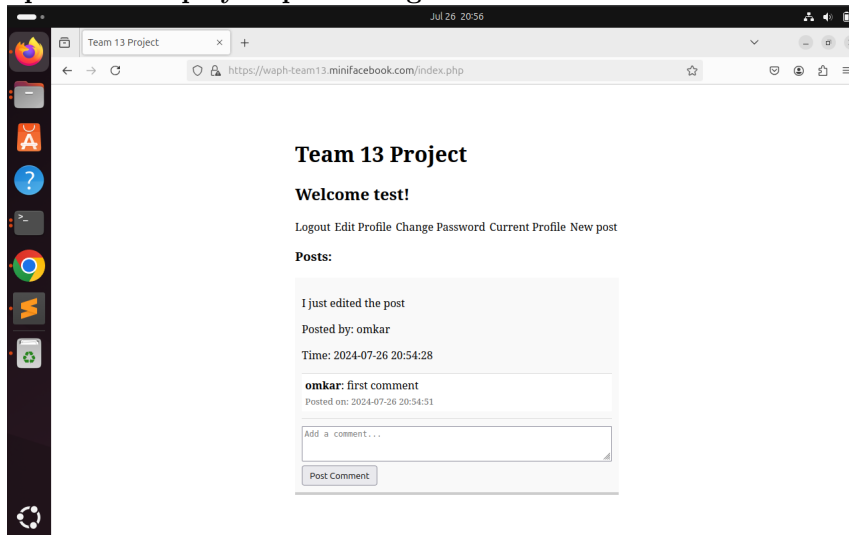
Sprint 2: Edit Post Functionality



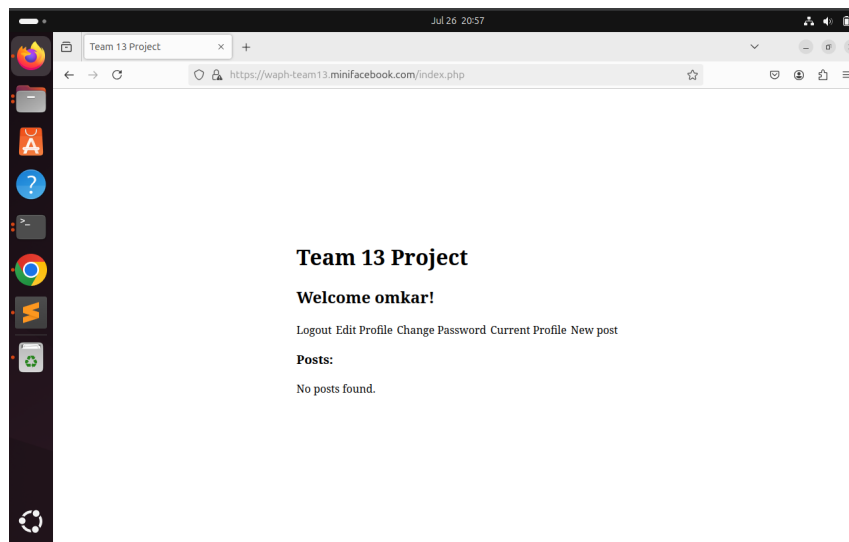
Sprint 2: Only the user that created the post can edit/delete the post



Sprint 2: Display of post being edited

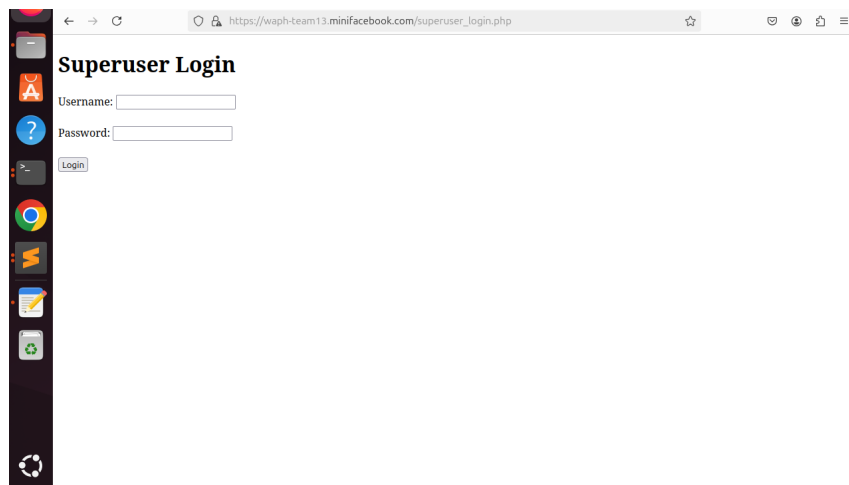


Sprint 2: Comment functionality

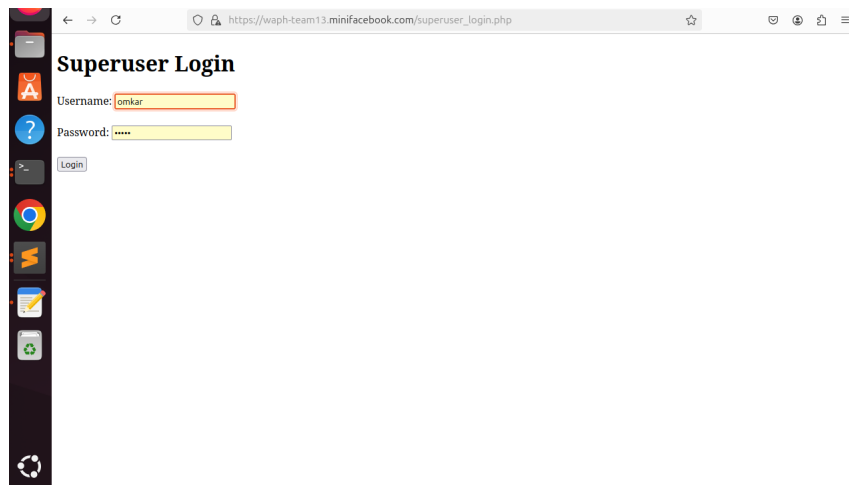


• Sprint 2: Delete user Functionality

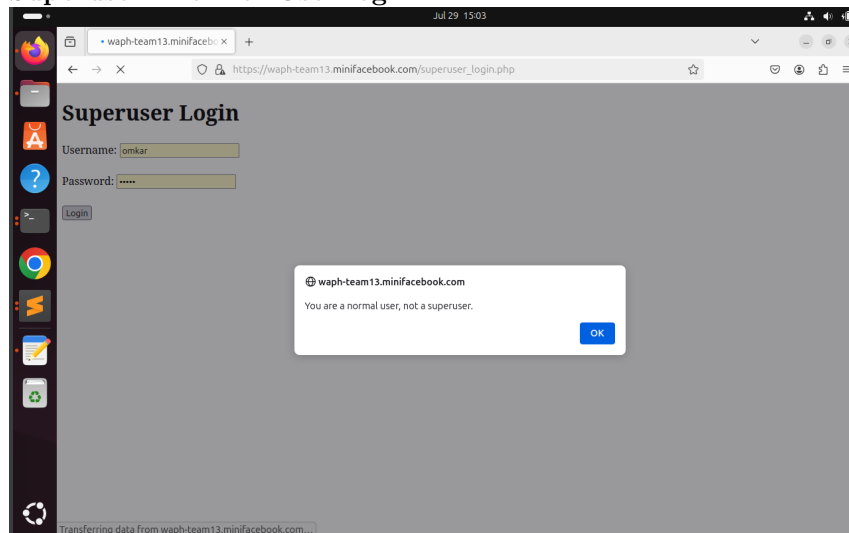
Superuser



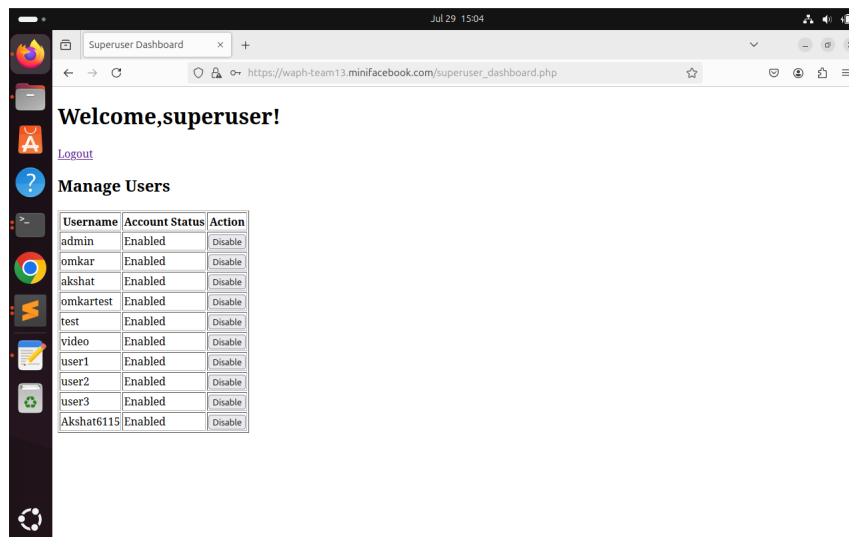
• Superuser: Superuser Login Page



• **Superuser: Normal User login**



• **Superuser: Error thrown if user is normal user**



• Superuser: Dashboard for Superuser

Software Process Management

(Start from Sprint 0, keep updating) **Sprint 0** Our team communicated through teams. We set-up timings everyday so that we can meet and discuss our progress and places where we were stuck. By helping each other through this methods we were able to complete the assignment succesfully.

Sprint 1 Due to facing issues with our laptops we began meeting in person. We set a schedule each day and time that we would meet. We would meet at those timing and try to resolve issues we were facing. Lastly we eneded up having to complete the whole spring on Omkar's laptop due to issues of code not working in Akshat's laptop.

Sprint 2Team communication was used by our group. Every day, we arrange a time to get together and talk about our accomplishments and any challenges that we faced. Through this way of working together assistance, we managed to do the work successfully.

Introduce how your team uses a software management process, e.g., Scrum, and how your teamwork collaborates.

Scrum process

Sprint 0

Duration: 07/07/2024-09/07/2024

Completed Tasks: Sprint 0 1. Certificate 2. team database 3. Index.html

Contributions:

1. Omkar, 2 commits, 1 hours, contributed in readme.md
2. Akshat, 2 commits, 1 hours, contributed in index.html
3. Member 3, x commits, y hours, contributed in xxx
4. Member 4, x commits, y hours, contributed in xxx

Sprint 1

We have created our team's sql server named 'waph_13' in which we have stored the required databases and the tables.

```
mysql> describe users;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| user_id    | int           | NO   | PRI | NULL    | auto_increment |
| username   | varchar(50)   | NO   |     | NULL    |               |
| password   | varchar(255)  | NO   |     | NULL    |               |
+-----+-----+-----+-----+-----+-----+
```

Sprint 1: Screenshot 1

```
mysql> describe posts
-> ;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| post_id    | int           | NO   | PRI | NULL    | auto_increment |
| user_id    | int           | YES  | MUL | NULL    |               |
| content     | text          | YES  |     | NULL    |               |
| timestamp  | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Sprint 1: Screenshot 2

Duration: 07/15/2024-07/21/2024

Completed Tasks:

1. Database design and implementation
2. User registration and login
3. Change password
4. Edit their profile, including name, additional email, phone

Contributions:

1. Omkar , 1 commits, 5 hours, login, signup, editprofile, index(home screen) functionalities.
2. Akshat, 1 commits, 4.5 hours, developed databases schema,logout, current profile functionalities.

Sprint 2 1. Updated the database design and implementation 2. Added a functionality for logged in users to add a post and comment in it. 3. Allow

logged in users to edit their posts

Contributions:

1. Omkar, 2 commits, 1 hours, contributed in readme.md
2. Akshat, 2 commits, 1 hours, contributed in index.html

Sprint Retrospection: __ (Introduction to Sprint Retrospection:

Working through the sprints is a continuous improvement process. Discussing the completed sprint can improve the next sprint walk through a much more efficient one. Sprint retrospection is done once a sprint is finished and the team is ready to start another sprint planning meeting. This discussion can take up to 1 hour depending on the ideal team size of 4 members. Discussing good things that happened during the sprint can improve the team's morale, good team collaboration, appreciating someone who did a fantastic job solving a blocker issue, work well-organized, and helping someone in need. This will improve the team's confidence and keep them motivated. As a team, we can discuss what has gone wrong during the sprint and come up with improvement points for the next sprints. Few points can be like, need to manage time well, need to prioritize the tasks properly and finish a task in time, incorrect design lead to multiple reviews and that wasted time during the sprint, team meetings were too long which consumed most of the effective work hours. We can mention every problem is in the sprint which is hindering the progress. Finally, this meeting should improve your next sprint drastically and understand the team dynamics well. Mention the bullet points and discuss how to solve it.)

Good	Could have been better	How to improve?
------	------------------------	-----------------

Good: Developing front end using html css was easy and well done.

Could have been better: We could have spent some more time ensure that we have all the necessities for the future sprints. When we began developing this part of the project we faced issues with connecting the sql database and our php code. We ended up having to restart everything from the code to installation of sql.

How to improve? In order to improve we must plan ahead so that in case such instance like this happen we aren't too late and are able to complete our required tasks in a timely manner.

Sprint 1

Duration: 07/15/2024-07/21/2024

Completed Tasks:

1. Database design and implementation
2. User registration and login
3. Change password
4. Edit their profile, including name, additional email, phone

Contributions:

1. Member 1, x commits, y hours, contributed in xxx
2. Member 2, x commits, y hours, contributed in xxx
3. Member 3, x commits, y hours, contributed in xxx
4. Member 4, x commits, y hours, contributed in xxx

Sprint Retrospection:

Good	Could have been better	How to improve?

Good: Developing the frontend editing part of the profile form. **Could have been better:** We could have taken a little longer to make sure we had everything we would need for the upcoming sprints. We encountered difficulties linking the SQL database and our PHP code when we started working on this section of the project. In the end, we had to start over from scratch with the SQL installation and the code. **How to improve:** We can follow a more detailed oriented approach while working on the code as well while connecting it to the databases.

Sprint 2 Duration: 07/21/2024-07/23/2024

Completed Tasks: 1. Updated Database design and implementation 2. Post, comment, add, edit creation for logged-in users 3. Updation of profile and posts for users. ... Contributions: Omkar, x 2 commits, 6 hours, contributed in form.php Akshat, 1 commit, 4 hours, contributed in profile.php

Sprint 3 Duration: 07/27/2024-07/30/2024

Completed Tasks: 1. Super User Login Page 2. Super User Dashboard

... Contributions: Omkar, 2 commits, 3 hours, contributed in superuser login-form Akshat, 1 commit, 3 hours, contributed in superuser dashboard # Appendix

Include the content (in text, not as images) of the SQL files and all source code of your PHP files (with the file name).

Source code: addcomment.php

```
<?php
session_start();
```

```

// Check if user is authenticated
if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== true) {
    header("Location: login.php");
    exit();
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Validate input
    $post_id = $_POST["post_id"];
    $user_id = $_SESSION["user_id"];
    $content = $_POST["content"];

    if (empty($content)) {
        die("Content is required! <a href='addcommentform.php?post_id=$post_id'>Go back to a
    }

    // Database connection
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        die("Database connection failed: " . $mysqli->connect_error);
    }

    // Insert comment into database
    $prepared_sql = "INSERT INTO comments (post_id, user_id, content) VALUES (?, ?, ?)";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {
        die("Prepare failed: " . $mysqli->error);
    }

    // Bind parameters and execute the statement
    $stmt->bind_param("iis", $post_id, $user_id, $content);
    if (!$stmt->execute()) {
        die("Execute failed: " . $stmt->error);
    }

    // Close statement and database connection
    $stmt->close();
    $mysqli->close();

    // Display success message
    echo "Comment added successfully!";
    echo '<br><a href="index.php">Back to Home</a>';
} else {
    // Invalid request method
    echo "Invalid request!";
}

```

?>

Source code: addcommentform.php

```
<?php
session_start();

// Check if user is authenticated
if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== true) {
    header("Location: login.php");
    exit();
}

if ($_SERVER["REQUEST_METHOD"] == "GET" && isset($_GET["post_id"])) {
    $post_id = $_GET["post_id"];
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Add Comment</title>
</head>
<body>
    <h2>Add Comment</h2>

    <form action="addcomment.php" method="post">
        <!-- Hidden input field to pass post_id to addcomment.php -->
        <input type="hidden" name="post_id" value="<?php echo htmlentities($post_id); ?>">

        <label for="content">Content:</label><br>
        <textarea id="content" name="content" rows="4" cols="50" required></textarea><br><br>

        <input type="submit" value="Submit">
    </form>
    <br><a href="index.php">Home Page</a>
</body>
</html>

<?php
} else {
    echo "Invalid request!";
}
?>
```

source code: addnewpost.php

```
<?php
```

```

session_start();

// Check if user is authenticated
if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== true) {
    header("Location: login.php"); // Redirect to login page if not logged in
    exit();
}

// Get user_id from session
$user_id = $_SESSION['user_id']; // Assuming you store user_id in the session

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Validate input
    $content = $_POST["content"];

    if (empty($content)) {
        die("Content is required! <a href='addnewpostform.php'>Go back to add post</a>");
    }

    // Database connection
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        die("Database connection failed: " . $mysqli->connect_error);
    }

    // Insert post into database
    $prepared_sql = "INSERT INTO posts (user_id, content) VALUES (?, ?)";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {
        die("Prepare failed: " . $mysqli->error);
    }

    $stmt->bind_param("is", $user_id, $content);
    if (!$stmt->execute()) {
        die("Execute failed: " . $stmt->error);
    }

    // Close statement and database connection
    $stmt->close();
    $mysqli->close();

    // Display success message
    echo "Post added successfully!";
    echo '<br><a href="addnewpostform.php">Add another post</a>';
} else {

```

```

        // Invalid request method
        echo "Invalid request!";
    }
    ?>

```

Source code: addnewpostform.php

```

<?php
    $username = $_POST["username"];
    $password = $_POST["password"];
    $name = $_POST["name"];
    $additional_email = $_POST["additional_email"];
    $email = $_POST["email"];
    $phone = $_POST["phone"];

    // Validate email format
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Invalid email format!";
        echo '<a href="registrationform.php">registration form</a>';
        exit;
    }

    if (userExists($username)) {
        echo "User already exists!";
        echo '<a href="registrationform.php">registration form</a>';
        exit;
    }

    if (isset($username) && isset($password) && isset($name) && isset($additional_email) &&
    {
        // Add user and fetch auto-generated user_id
        $user_id = addNewUser($username, $password);

        if ($user_id !== false) {
            if (addUserProfile($user_id, $name, $additional_email, $phone, $email)) {
                echo "Registration succeeded!";
                echo '<a href="form2.php">Log In</a>';

            } else {
                echo "Failed to add user profile!";
                echo '<a href="registrationform.php">registration form</a>';
            }
        } else {
            echo "Registration failed!";
        }
    }

```

```

        echo '<a href="registrationform.php">registration form</a>';
    }
} else {
    echo "Incomplete data provided!";
    echo '<a href="registrationform.php">registration form</a>';
}

function addNewUser($username, $password) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');

    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return false;
    }

    $prepared_sql = "INSERT INTO users (username, password) VALUES (?, MD5(?))";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {
        printf("Prepare failed: %s\n", $mysqli->error);
        return false;
    }

    $stmt->bind_param("ss", $username, $password);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        return false;
    }

    $user_id = $mysqli->insert_id;
    $stmt->close();
    return $user_id;
}

function addUserProfile($user_id, $name, $additional_email, $phone, $email) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');

    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return false;
    }

    $prepared_sql = "INSERT INTO profiles (user_id, name, additional_email, phone, email) VALUES (?, ?, ?, ?, ?)";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {

```

```

        printf("Prepare failed: %s\n", $mysqli->error);
        return false;
    }

    $stmt->bind_param("issss", $user_id, $name, $additional_email, $phone, $email);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        return false;
    }

    $stmt->close();
    return true;
}

function userExists($username) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return true;
    }

    $sql = "SELECT 1 FROM users WHERE username = ?";
    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        printf("Prepare failed: %s\n", $mysqli->error);
        return true;
    }

    $stmt->bind_param("s", $username);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        $stmt->close();
        return true;
    }

    $result = $stmt->get_result();
    $exists = $result->num_rows > 0;
    $stmt->close();
    return $exists;
}

```

?>

Source code: addnewuser.php

```

<?php
    $username = $_POST["username"];
    $password = $_POST["password"];
    $name = $_POST["name"];
    $additional_email = $_POST["additional_email"];
    $email = $_POST["email"];
    $phone = $_POST["phone"];

    // Validate email format
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Invalid email format!";
        echo '<a href="registrationform.php">registration form</a>';
        exit;
    }

    if (userExists($username)) {
        echo "User already exists!";
        echo '<a href="registrationform.php">registration form</a>';
        exit;
    }

    if (isset($username) && isset($password) && isset($name) && isset($additional_email) &&
    {
        // Add user and fetch auto-generated user_id
        $user_id = addNewUser($username, $password);

        if ($user_id !== false) {
            if (addUserProfile($user_id, $name, $additional_email, $phone, $email)) {
                echo "Registration succeeded!";
                echo '<a href="form2.php">Log In</a>';

            } else {
                echo "Failed to add user profile!";
                echo '<a href="registrationform.php">registration form</a>';
            }
        } else {
            echo "Registration failed!";
            echo '<a href="registrationform.php">registration form</a>';
        }
    } else {
        echo "Incomplete data provided!";
        echo '<a href="registrationform.php">registration form</a>';
    }
}

```

```

function addNewUser($username, $password) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');

    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return false;
    }

    $prepared_sql = "INSERT INTO users (username, password) VALUES (?, MD5(?))";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {
        printf("Prepare failed: %s\n", $mysqli->error);
        return false;
    }

    $stmt->bind_param("ss", $username, $password);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        return false;
    }

    $user_id = $mysqli->insert_id;
    $stmt->close();
    return $user_id;
}

function addUserProfile($user_id, $name, $additional_email, $phone, $email) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');

    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return false;
    }

    $prepared_sql = "INSERT INTO profiles (user_id, name, additional_email, phone, email) VALUES (?, ?, ?, ?, ?)";
    $stmt = $mysqli->prepare($prepared_sql);
    if (!$stmt) {
        printf("Prepare failed: %s\n", $mysqli->error);
        return false;
    }

    $stmt->bind_param("issss", $user_id, $name, $additional_email, $phone, $email);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        return false;
    }
}

```

```

    }

    $stmt->close();
    return true;
}

function userExists($username) {
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return true;
    }

    $sql = "SELECT 1 FROM users WHERE username = ?";
    $stmt = $mysqli->prepare($sql);
    if (!$stmt) {
        printf("Prepare failed: %s\n", $mysqli->error);
        return true;
    }

    $stmt->bind_param("s", $username);
    if (!$stmt->execute()) {
        printf("Execute failed: %s\n", $stmt->error);
        $stmt->close();
        return true;
    }

    $result = $stmt->get_result();
    $exists = $result->num_rows > 0;
    $stmt->close();
    return $exists;
}

```

?>

Source code: changepassword.php

```

<?php
    $username = $_POST["username"];
    $password = $_POST["password"];
    if (isset($username) and isset($password)){

        if(changepassword($username,$password))
        {
            echo " Your password has been changed!";
        }
    }

```

```

else
{
    echo "Registration failed!";
}
}else {
    echo "No username/password provided!";
}

function changepassword($username, $password)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return FALSE;
    }

    $hashed_password = md5($password);

    $prepared_sql = "UPDATE users SET password = ? WHERE username = ?;";
    $stmt = $mysqli->prepare($prepared_sql);
    // Binding parameters
    $stmt->bind_param("ss", $hashed_password, $username);
    $stmt->execute();

    if ($mysqli->affected_rows == 1)
        return TRUE;
    return FALSE;
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Change Password</title>
    <link rel="stylesheet" href="minifbstyle.css">
</head>
<body>
<div class="container">
    <header>
        <a href="index.php">Home Page</a>
        <a href="logout.php">Logout</a>
        <a href="changepasswordform.php">back</a>

    </div>

```

```

    </header>
</div>
</body>
</html>

```

Source code: [changepasswordform.php](#)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Change Password</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      margin: 0;
      padding: 0;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
    }
    .container {
      max-width: 400px;
      width: 100%;
      background-color: #fff;
      padding: 30px;
      border-radius: 8px;
      box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
      text-align: center;
      color: #333;
    }
    h1 {
      color: #000; /* Black heading color */
      margin-bottom: 20px;
    }
    .button {
      width: 50%; /* Adjusted button width */
      padding: 8px; /* Smaller padding for compact size */
      background-color: #000; /* Black button background */
      color: #fff;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      font-size: 14px; /* Smaller font size */
      transition: background-color 0.3s ease;
    }
  </style>

```

```

    }
    .button:hover {
        background-color: #333; /* Darker shade on hover */
    }
    .home-link {
        display: block;
        text-decoration: none;
        color: #000; /* Black link color */
        margin-top: 20px;
        font-size: 16px;
    }
    .home-link:hover {
        text-decoration: underline;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>Change Password</h1>
        <form action="changepassword.php" method="POST" class="form login">
            <input type="hidden" name="username" value="<?php echo htmlspecialchars($_SESSION['username'])"/>
            Password: <input type="password" class="text_field" name="password" /> <br>
            <button class="button" type="submit">Submit</button>
        </form>
        <a href="index.php" class="home-link">Home Page</a>
    </div>
</body>
</html>

```

Source code: database-account.sql

```

create database waph_13;
CREATE USER 'waph_team13'@'localhost' IDENTIFIED BY 'password';
GRANT ALL ON waph_13* TO 'waph_team13'@'localhost';

```

Source-code: database-data.sql

```

CREATE TABLE users(
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50),
    password VARCHAR(255) NOT NULL,
    is_disabled BOOLEAN DEFAULT FALSE
);
CREATE TABLE profiles(
    user_id INT PRIMARY KEY,
    name VARCHAR(100),

```

```

        additional_email VARCHAR(255),
        email VARCHAR(255) UNIQUE NOT NULL,
        phone VARCHAR(20),
        FOREIGN KEY (user_id) REFERENCES users(user_id)
    );

CREATE TABLE posts(
    post_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE comments(
    comment_id INT PRIMARY KEY AUTO_INCREMENT,
    post_id INT,
    user_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES posts(post_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE superusers(
    superuser_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);

CREATE TABLE chat_messages(
    message_id INT PRIMARY KEY AUTO_INCREMENT,
    sender_id INT,
    receiver_id INT,
    content TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sender_id) REFERENCES users(user_id),
    FOREIGN KEY (receiver_id) REFERENCES users(user_id)
);

INSERT INTO superusers (username,password) VALUES ('admin',md5('admin'));
INSERT INTO users (username,password) VALUES ('admin',md5('admin'));

GRANT ALL ON users TO 'waph_team13'@'localhost';
GRANT ALL ON profiles TO 'waph_team13'@'localhost';
GRANT ALL ON posts TO 'waph_team13'@'localhost';
GRANT ALL ON comments TO 'waph_team13'@'localhost';
GRANT ALL ON superusers TO 'waph_team13'@'localhost';

```

```
GRANT ALL ON chat_messages TO 'waph_team13'@'localhost';
```

Source code: editprofile.php

```
<?php
```

```
$username = $_POST["username"];
$name = $_POST["name"];
$additional_email = $_POST["additional_email"];
$phone = $_POST["phone"];

// Check if the form is submitted
if (isset($username) && isset($name) && isset($additional_email) && isset($phone)) {
    if (updateProfile($username, $name, $additional_email, $phone)) {
        echo "Profile updated successfully!";
    } else {
        echo "Failed to update profile!";
    }
} else {
    echo "Invalid data provided!";
}

function updateProfile($username, $name, $additional_email, $phone)
{

    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        return FALSE;
    }

    $query = "SELECT user_id FROM users WHERE username = ?";
    $stmt = $mysqli->prepare($query);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows == 0) {
        echo "Error: User not found!";
        return false;
    }
}
```

```

$row = $result->fetch_assoc();
$user_id = $row['user_id'];

// Update the profile information in the database
$query = "UPDATE profiles SET name = ?, additional_email = ?, phone = ? WHERE user_id = ";
$stmt = $mysqli->prepare($query);
$stmt->bind_param("sssi", $name, $additional_email, $phone, $user_id);
$stmt->execute();

if ($stmt->affected_rows == 1) {
    echo "Profile updated successfully!";
    return true;
} else {
    echo "Failed to update profile!";
    return false;
}
}

```

?>

Source code: editprofileform.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>WAPH-Edit Profile</title>
<style>
body {
    font-family: Arial, sans-serif;
    background-color: #f9f9f9; /* Light gray background */
    color: #333; /* Dark gray text color */
}
h1, h2 {
    color: #000; /* Black heading color */
    text-align: center;
}
.form {
    max-width: 300px; /* Adjust form width as needed */
    margin: 20px auto; /* Center align the form */
    padding: 20px;
    background: #ffffe0; /* Light yellow background */
    border: 2px solid #000; /* Black border */
    border-radius: 5px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); /* Shadow effect */
}
.text_field {

```

```

        width: 100%;
        padding: 10px;
        margin-bottom: 10px;
        border: 1px solid #000; /* Black border */
        border-radius: 5px;
        box-sizing: border-box;
    }
    .button {
        width: 100%;
        padding: 10px;
        background-color: #007bff; /* Blue button background */
        color: #fff; /* White button text color */
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    .button:hover {
        background-color: #0056b3; /* Darker blue on hover */
    }
    .home-link {
        display: block;
        text-align: center;
        margin-top: 20px;
        text-decoration: none;
        color: #007bff; /* Blue link color */
    }
    .home-link:hover {
        text-decoration: underline;
    }
</style>
</head>
<body>
    <?php
    session_start();
    ?>
    <h1>Edit Profile, WAPH</h1>
    <h2>Welcome <?PHP echo htmlentities($_SESSION['username']); ?> !</h2>

    <form action="editprofile.php" method="POST" class="form edit-profile">
        <input type="hidden" name="username" value="<?php echo $_SESSION['username']; ?>">
        Name: <input type="text" class="text_field" name="name" /> <br>
        Additional Email: <input type="email" class="text_field" name="additional_email" /> <br>
        Phone: <input type="tel" class="text_field" name="phone" /> <br>
        <!-- Add more fields as needed -->
        <button class="button" type="submit">Save Changes</button>
    </form>

```

```

    <a href="index.php" class="home-link">Home Page</a>
</body>
</html>

```

Source code: form2.php

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>WAPH-Login page</title>
  <style>
    body {
      color: #000; /* Black text color */
    }
    .container {
      background-color: #f2f2f2; /* Light gray background */
      padding: 20px;
      border-radius: 10px;
      margin: 20px auto;
      width: 400px; /* Adjust width as needed */
    }
    .header {
      text-align: center;
      margin-bottom: 20px;
    }
    .header h1 {
      color: #000; /* Black text color */
    }
    .form-group label {
      color: #000; /* Black label color */
    }
    .btn-primary {
      background-color: #007bff; /* Blue button background */
      color: #fff; /* White button text color */
      border: none;
      padding: 10px 20px;
      border-radius: 5px;
      cursor: pointer;
      display: block;
      margin: 10px auto;
    }
    .btn-primary:hover {
      background-color: #0056b3; /* Darker blue on hover */
    }
    .btn-secondary {

```

```

        background-color: #007bff; /* Blue button background */
        color: #fff; /* White button text color */
        border: none;
        padding: 10px 20px;
        border-radius: 5px;
        cursor: pointer;
        display: block;
        margin: 10px auto;
    }
    .btn-secondary:hover {
        background-color: #0056b3; /* Darker blue on hover */
    }
</style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Please Login</h1>
            <h2>Team 13</h2>
        </div>
        <form action="index.php" method="POST" class="form login">
            <div class="form-group">
                <label for="username">Username:</label>
                <input type="text" class="form-control" id="username" name="username" placeholder="Username">
            </div>
            <div class="form-group">
                <label for="password">Password:</label>
                <input type="password" class="form-control" id="password" name="password" placeholder="Password">
            </div>
            <!-- Button Group -->
            <div class="button-group" style="text-align: center;">
                <button type="submit" class="btn btn-primary">Log In</button>
                <button type="button" onclick="window.location.href='registrationform.php';" class="btn btn-secondary">Register</button>
            </div>
        </form>
    </div>
</body>
</html>

```

Source code: index.php

```

<?php
$lifetime = 15 * 60;
$path = "/";
$domain = "10.0.3.15";
$secure = TRUE;

```

```

$httponly = TRUE;
session_set_cookie_params($lifetime, $path, $domain, $secure, $httponly);
session_start();

if (isset($_POST["username"]) && isset($_POST["password"])) {
    if (checklogin_mysql($_POST["username"], $_POST["password"])) {
        $_SESSION['authenticated'] = TRUE;
        $_SESSION['username'] = $_POST["username"];
        $_SESSION['browser'] = $_SERVER["HTTP_USER_AGENT"];
    } else {
        session_destroy();
        echo "<script>alert('Invalid username/password');window.location='form2.php';</script>";
        die();
    }
}

if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== TRUE) {
    session_destroy();
    echo "<script>alert('You have not logged in. Please login first!');</script>";
    header("Refresh: 0; url=form2.php");
    die();
}

if ($_SESSION["browser"] != $_SERVER["HTTP_USER_AGENT"]) {
    session_destroy();
    echo "<script>alert('Session hijacking attack is detected!');</script>";
    header("Refresh:0; url=form2.php");
    die();
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_POST['comment'])) {
        $postId = $_POST['post_id'];
        $comment = $_POST['comment'];

        if (!empty($comment)) {
            addComment($postId, $_SESSION['username'], $comment);
        }
    } elseif (isset($_POST['edit_post'])) {
        $postId = $_POST['post_id'];
        $content = $_POST['content'];
        editPost($postId, $content);
    } elseif (isset($_POST['delete_post'])) {
        $postId = $_POST['post_id'];
        deletePost($postId);
    }
}

```

```

}

function checklogin_mysql($username, $password)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }
    $sql = "SELECT * FROM users WHERE username=? AND password = md5(?)";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("ss", $username, $password);
    $stmt->execute();
    $result = $stmt->get_result();
    return $result->num_rows >= 1;
}

function getPosts()
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }
    $sql = "SELECT posts.post_id, profiles.name, posts.content, posts.timestamp
            FROM posts
            INNER JOIN profiles ON posts.user_id = profiles.user_id
            ORDER BY posts.timestamp DESC";
    $result = $mysqli->query($sql);
    if ($result->num_rows > 0) {
        while ($row = $result->fetch_assoc()) {
            echo "<div class='post'>";
            echo "<div class='post-content'>";
            echo "<p>" . htmlspecialchars($row['content']) . "</p>";
            echo "<p>Posted by: " . htmlspecialchars($row['name']) . "</p>";
            echo "<p>Time: " . htmlspecialchars($row['timestamp']) . "</p>";
            echo "</div>";

            // Check if the logged-in user is the author of the post
            if ($_SESSION['username'] === htmlspecialchars($row['name'])) {
                // Edit and delete buttons
                echo "<div class='post-actions'>";
                echo "<button class='edit-btn' onclick='editPost(" . $row['post_id'] . ")'>";
                echo "<form method='POST' style='display:inline;' action='index.php'>";
                echo "<input type='hidden' name='post_id' value='" . htmlspecialchars($row[
                echo "<button type='submit' name='delete_post'>Delete</button>";
            }
        }
    }
}

```

```

        echo "</form>";
        echo "</div>";
    }

    // Display comments
    displayComments($row['post_id']);

    // Comment form
    echo "<div class='comment-form'>";
    echo "<form method='POST' action='index.php'>";
    echo "<input type='hidden' name='post_id' value='" . htmlspecialchars($row['post_id']) . "'>";
    echo "<textarea name='comment' rows='3' placeholder='Add a comment...' required='required'>";
    echo "<button type='submit'>Post Comment</button>";
    echo "</form>";
    echo "</div>";

    echo "</div>";
}
} else {
    echo "<p>No posts found.</p>";
}
}

function displayComments($postId)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }

    $sql = "SELECT comments.content, profiles.name, comments.timestamp
            FROM comments
            INNER JOIN profiles ON comments.user_id = profiles.user_id
            WHERE post_id = ?
            ORDER BY comments.timestamp DESC";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("i", $postId);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows > 0) {
        while ($row = $result->fetch_assoc()) {
            echo "<div class='comment'>";
            echo "<p><strong>" . htmlspecialchars($row['name']) . "</strong>: " . htmlspecialchars($row['content']) . "</p>";
            echo "<p><small>Posted on: " . htmlspecialchars($row['timestamp']) . "</small></p>";
            echo "</div>";
        }
    }
}

```

```

    } else {
        echo "<p>No comments yet.</p>";
    }
}

function addComment($postId, $username, $comment)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }

    // Retrieve user_id
    $sql = "SELECT user_id FROM users WHERE username = ?";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $result = $stmt->get_result();
    $user = $result->fetch_assoc();
    $userId = $user['user_id'];

    // Insert comment
    $sql = "INSERT INTO comments (post_id, user_id, content, timestamp) VALUES (?, ?, ?, NOW())";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("iis", $postId, $userId, $comment);
    $stmt->execute();
}

function editPost($postId, $content)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }

    // Update post
    $sql = "UPDATE posts SET content = ? WHERE post_id = ?";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("si", $content, $postId);
    $stmt->execute();
}

function deletePost($postId)
{

```

```

$mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
if ($mysqli->connect_errno) {
    printf("Database connection failed: %s\n", $mysqli->connect_error);
    exit();
}

// Begin a transaction
$mysqli->begin_transaction();

try {
    // Delete all comments associated with the post
    $sql = "DELETE FROM comments WHERE post_id = ?";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("i", $postId);
    $stmt->execute();

    // Delete the post itself
    $sql = "DELETE FROM posts WHERE post_id = ?";
    $stmt = $mysqli->prepare($sql);
    $stmt->bind_param("i", $postId);
    $stmt->execute();

    // Commit the transaction
    $mysqli->commit();
} catch (Exception $e) {
    // Rollback the transaction on failure
    $mysqli->rollback();
    printf("Error deleting post and comments: %s\n", $e->getMessage());
}
}

?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Team 13 Project</title>
    <link rel="stylesheet" href="minifbstyle.css">
    <style>
        .user-info a {
            color: #000000;
            text-decoration: none;
            margin-right: 2px;
            transition: color 0.1s;
        }
    </style>

```

```

body, html {
    height: 100%;
    margin: 0;
    display: flex;
    justify-content: center;
    align-items: center;
}
.user-info a:hover {
    color: #000000;
}
.main-content h3 {
    color: #000000;
}
.post {
    border-bottom: 4px solid #ccc;
    padding: 10px;
    margin-bottom: 10px;
    background-color: #f9f9f9;
}
.post-content {
    margin-bottom: 10px;
}
.post:hover {
    background-color: #f0f0f0;
}
.comment-form {
    border-top: 1px solid #ddd;
    padding-top: 10px;
    margin-top: 10px;
}
.comment-form textarea {
    width: 100%;
    box-sizing: border-box;
}
.comment-form button {
    margin-top: 5px;
    padding: 5px 10px;
}
.comment {
    border-top: 1px solid #ddd;
    padding: 5px;
    margin-top: 5px;
    background-color: #fff;
}
.comment p {
    margin: 0;
}

```

```

    }
    .comment p small {
        color: #666;
    }
    .post-actions {
        margin-top: 10px;
    }
    .post-actions button {
        margin-right: 10px;
    }
</style>
<script>
    function editPost(postId) {
        var content = prompt("Edit your post content:");
        if (content) {
            var form = document.createElement('form');
            form.method = 'POST';
            form.action = 'index.php';

            var postIdInput = document.createElement('input');
            postIdInput.type = 'hidden';
            postIdInput.name = 'post_id';
            postIdInput.value = postId;
            form.appendChild(postIdInput);

            var contentInput = document.createElement('input');
            contentInput.type = 'hidden';
            contentInput.name = 'content';
            contentInput.value = content;
            form.appendChild(contentInput);

            var editPostInput = document.createElement('input');
            editPostInput.type = 'hidden';
            editPostInput.name = 'edit_post';
            form.appendChild(editPostInput);

            document.body.appendChild(form);
            form.submit();
        }
    }
</script>
</head>
<body>
<div class="container">
    <header>
        <h1 class="title">Team 13 Project</h1>

```

```

        <div class="user-info">
            <h2>Welcome <?php echo htmlspecialchars($_SESSION['username']); ?>!</h2>
            <a href="logout.php">Logout</a>
            <a href="editprofileform.php">Edit Profile</a>
            <a href="changepasswordform.php">Change Password</a>
            <a href="profile.php">Current Profile</a>
            <a href="addnewpostform.php">New post</a>
        </div>
    </header>
    <section class="main-content">
        <h3>Posts:</h3>
        <?php getPosts(); ?>
    </section>
</div>
</body>
</html>

```

Source code: logout.php

```

<?php
    session_start();
    session_destroy();
?>
<p> You have been logged out</p>
<a href="form2.php">Click here to login again</a>

```

Source code: minifbstyle.css

```

/* Reset and base styles */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: Arial, sans-serif;
    background-color: #f0f2f5;
}

.container {
    max-width: 800px;
    margin: 20px auto;
    padding: 20px;
    background-color: #fff;
    border: 1px solid #dddfe2;
    border-radius: 5px;
}

```

```

}

header {
    padding-bottom: 20px;
    border-bottom: 1px solid #dddfe2;
}

header h1 {
    font-size: 24px;
    font-weight: bold;
    color: #1877f2;
    margin-top: 0;
}

.header h2,
.user-info h2 {
    font-size: 18px;
    color: #1877f2;
    margin-top: 0;
}

.user-info a {
    color: #1877f2;
    text-decoration: none;
    margin-right: 10px;
}

/* Form and button styles */
.form-group,
.login-form-group {
    margin-bottom: 20px;
}

.form-group label,
.login-form-group label {
    font-size: 14px;
    color: #333;
}

.form-control,
.login-form-control {
    width: 100%;
    padding: 10px;
    font-size: 16px;
    border: 1px solid #dddfe2;
    border-radius: 5px;
}

```

```

}

.btn-primary,
.login-btn-primary {
    background-color: #1877f2;
    color: #fff;
    padding: 10px 20px;
    font-size: 16px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

.btn-primary:hover,
.login-btn-primary:hover {
    background-color: #166fe5;
}

/* Post styles */
.post {
    background-color: #fff;
    border: 1px solid #dddfe2;
    border-radius: 5px;
    padding: 15px;
    margin-bottom: 20px;
}

.post h3 {
    font-size: 18px;
    margin-bottom: 10px;
}

.post p {
    font-size: 16px;
    margin-bottom: 5px;
}

.post p:last-child {
    margin-bottom: 0;
}

.post p.timestamp {
    color: #888;
}

.post p.user {

```

```

        color: #555;
    }

    .post p.content {
        margin-top: 10px;
    }

```

Source code: profile.php

```

<?php
session_start();

if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== true) {
    header("Location: login.php"); // Redirect to login page if not logged in
    exit();
}

$username = $_SESSION['username'];

$mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
if ($mysqli->connect_errno) {
    echo "Failed to connect to MySQL: " . $mysqli->connect_error;
    exit();
}

$query = "SELECT * FROM users INNER JOIN profiles ON users.user_id = profiles.user_id WHERE";
$stmt = $mysqli->prepare($query);
$stmt->bind_param("s", $username);
$stmt->execute();
$result = $stmt->get_result();
if ($result->num_rows > 0) {
    $row = $result->fetch_assoc();
    $name = $row['name'];
    $email = $row['email'];
    $additional_email = $row['additional_email'];
    $phone = $row['phone'];
} else {
    echo "No profile found for the logged-in user.";
}

$stmt->close();
$mysqli->close();
?>

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<title>Profile</title>
<style>
    body {
        font-family: Arial, sans-serif;
        background-color: #f9f9f9;
        margin: 0;
        padding: 0;
    }
    .profile-container {
        max-width: 400px;
        margin: 100px auto;
        padding: 100px;
        border: 1px solid #ccc;
        border-radius: 10px;
        background-color: #fff;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    }
    .profile-container h2 {
        margin-top: 0;
        color: #007bff; /* Blue heading color */
        text-align: center;
    }
    .profile-info {
        margin-bottom: 10px;
    }
    .profile-info label {
        font-weight: bold;
    }
    .profile-info span {
        color: #555; /* Dark gray text color */
    }
    .home-link {
        display: block;
        text-align: center;
        margin-top: 20px;
        text-decoration: none;
        color: #007bff; /* Blue link color */
    }
    .home-link:hover {
        text-decoration: underline;
        color: #000000;
    }
</style>
</head>
<body>

```

```

<div class="profile-container">
  <h2>Profile Details</h2>
  <div class="profile-info">
    <label>Name:</label>
    <span><?php echo $name; ?></span>
  </div>
  <div class="profile-info">
    <label>Email:</label>
    <span><?php echo $email; ?></span>
  </div>
  <div class="profile-info">
    <label>Additional Email:</label>
    <span><?php echo $additional_email; ?></span>
  </div>
  <div class="profile-info">
    <label>Phone:</label>
    <span><?php echo $phone; ?></span>
  </div>
  <a href="index.php" class="home-link">Home Page</a>
</div>

</body>
</html>

```

Source code: registrationform.php

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Sign Up</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      color: #333;
      margin: 0;
      padding: 0;
    }
    h1 {
      text-align: center;
      color: #000000;
    }
    #digit-clock {
      text-align: center;
      font-size: 16px;
    }

```

```

        margin-bottom: 20px;
    }
    .form {
        max-width: 400px;
        margin: 20px auto;
        background: #fff;
        padding: 20px;
        border-radius: 5px;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    }
    .text_field {
        width: 60%;
        padding: 10px;
        margin-bottom: 10px;
        border: 1px solid #ccc;
        border-radius: 5px;
        box-sizing: border-box;
    }
    .button {
        width: 100%;
        padding: 10px;
        background-color: #000000;
        color: #fff;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    .button:hover {
        background-color: #0056b3;
    }
    .home-link {
        display: block;
        text-align: center;
        margin-top: 20px;
        text-decoration: none;
        color: #000000; /* Blue link color */
    }
    .home-link:hover {
        text-decoration: underline;
    }
</style>
<script>
function validateForm() {
    // Client-side validation logic
    var username = document.forms["registrationForm"]["username"].value;
    var password = document.forms["registrationForm"]["password"].value;

```

```

        var email = document.forms["registrationForm"]["email"].value;
        // Add more validation checks as needed

        if (username == "" || password == "" || email == "") {
            alert("All fields are required");
            return false;
        }
        // Add more client-side validation checks as needed
        return true;
    }
</script>
</head>
<body>
    <h1>Team 13, Project: User Registration</h1>
    <form action="addnewuser.php" method="POST" class="form login">
        Username:<input type="text" class="text_field" name="username" /> <br>
        Password: <input type="password" class="text_field" name="password" /> <br>
        name:<input type="text" class="text_field" name="name" /> <br>
        additional_email: <input type="text" class="text_field" name="additional_email" /> <br>
        email:<input type="text" class="text_field" name="email" /> <br>
        phone: <input type="text" class="text_field" name="phone" /> <br>
        <button class="button" type="submit">Submit</button>
        <a href="form2.php" class="home-link">Login Page</a>
    </form>

</body>
</html>

```

Source code: superuser_dashboard.php

```

<?php
session_start();

// Check if the user is authenticated and is a superuser
if (!isset($_SESSION['authenticated']) || $_SESSION['authenticated'] !== TRUE || !isset($_SESSION['superuser'])) {
    session_destroy();
    echo "<script>alert('You do not have permission to access this page.');"window.location='';
    die();
}

// Connect to the database
$mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
if ($mysqli->connect_errno) {
    printf("Database connection failed: %s\n", $mysqli->connect_error);
    exit();
}

```

```

// Fetch all users
$sql = "SELECT * FROM users";
$result = $mysqli->query($sql);
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Superuser Dashboard</title>
</head>
<body>
<div class="container">
    <h1>Welcome,<?php echo htmlspecialchars($_SESSION['username']); ?>!</h1>
    <a href="logout.php">Logout</a>

    <h2>Manage Users</h2>
    <table border="1">
        <thead>
            <tr>
                <th>Username</th>
                <th>Account Status</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody>
            <?php
            if ($result->num_rows > 0) {
                while ($row = $result->fetch_assoc()) {
                    echo "<tr>";
                    echo "<td>" . htmlspecialchars($row['username']) . "</td>";
                    echo "<td>" . ($row['is_disabled'] ? 'Disabled' : 'Enabled') . "</td>";
                    echo "<td>";
                    if ($row['is_disabled']) {
                        echo "<form action='enableuser.php' method='post' style='display:inline-block'>";
                        echo "<input type='hidden' name='user_id' value='" . $row['user_id'] . "'>";
                        echo "<input type='submit' value='Enable'>";
                        echo "</form>";
                    } else {
                        echo "<form action='disableuser.php' method='post' style='display:inline-block'>";
                        echo "<input type='hidden' name='user_id' value='" . $row['user_id'] . "'>";
                        echo "<input type='submit' value='Disable'>";
                        echo "</form>";
                    }
                    echo "</td>";
                }
            }
        </tbody>
    </table>
</div>

```

```

        echo "</tr>";
    }
    } else {
        echo "<tr><td colspan='3'>No users found.</td></tr>";
    }
    ?>
</tbody>
</table>
</div>
</body>
</html>

```

Source code: superuser_login.php

```

<?php
$lifetime = 15 * 60;
$path = "/";
$domain = "10.0.3.15";
$secure = TRUE;
$httponly = TRUE;
session_set_cookie_params($lifetime, $path, $domain, $secure, $httponly);
session_start();

if (isset($_POST["username"]) && isset($_POST["password"])) {
    if (checkSuperuserLogin($_POST["username"], $_POST["password"])) {
        $_SESSION['authenticated'] = TRUE;
        $_SESSION['username'] = $_POST["username"];
        $_SESSION['is_superuser'] = TRUE;
        $_SESSION['browser'] = $_SERVER["HTTP_USER_AGENT"];
        header("Location: superuser_dashboard.php"); // Redirect to superuser dashboard
        exit();
    } else {
        session_destroy();
        echo "<script>alert('Invalid username/password or you are not a superuser.');"window
        die();
    }
}

function checkSuperuserLogin($username, $password)
{
    $mysqli = new mysqli('localhost', 'waph13', 'password', 'waph');
    if ($mysqli->connect_errno) {
        printf("Database connection failed: %s\n", $mysqli->connect_error);
        exit();
    }
}

```

```

        // Check if the user is in the superusers table
        $sql = "SELECT * FROM superusers WHERE username=? AND password = md5(?)";
        $stmt = $mysqli->prepare($sql);
        $stmt->bind_param("ss", $username, $password);
        $stmt->execute();
        $result = $stmt->get_result();
        if ($result->num_rows >= 1) {
            return TRUE;
        }

        // If not found in superusers, check if it's a normal user
        $sql = "SELECT * FROM users WHERE username=? AND password = md5(?)";
        $stmt = $mysqli->prepare($sql);
        $stmt->bind_param("ss", $username, $password);
        $stmt->execute();
        $result = $stmt->get_result();
        if ($result->num_rows >= 1) {
            echo "<script>alert('You are a normal user, not a superuser.');

```